RITE: An IDE for Assurance and Certification of Software and Systems

Baoluo Meng*, Saswata Paul*, Sarat Chandra Varanasi*, Christopher Alexander*, Eric Mertens†, Abha Moitra*, Kit Siu*, Michael Durling*
*GE Aerospace Research, Niskayuna, NY, USA
†Galois Inc., Portland, OR, USA
Email: *{baoluo.meng, saswata.paul, saratchandra.varanasi, christopher.alexander, abha.moitra, siu, durling}@geaerospace.com
Email: †emertens@galois.com

2025 AIAA DATC/IEEE 44th Digital Avionics Systems Conference (DASC) | 979-8-3315-2519-4/25/$31.00 ©2025 IEEE | DOI: 10.1109/DASC66011.2025.11257205

***Abstract*—Safety-critical airborne software and systems must undergo rigorous certification before they are approved for deployment, which involves the evaluation of a vast amount of data generated during the development and verification lifecycle. Such a vast amount of heterogeneous data from different sources can be very difficult to manage and evaluate efficiently. We present RACK Integrated Certification Environment (RITE), an open-source toolchain based on Eclipse. RITE utilizes the Semantic Application Design Language (SADL), a formal language designed to assist engineers in developing certification arguments that demonstrate product compliance with standards such as DO-178C and ARP4754. It supports the ingestion of software and system development artifacts from heterogeneous sources into a curated semantic database supported by the Rapid Assurance Curation Kit (RACK). It also provides capabilities for automated construction, visualization, and assessment of assurance cases and compliance reports. RITE integrates 1) an ontology authoring tool to generate formal semantics to normalize heterogeneous data; 2) a semantic triplestore to curate normalized data; 3) an automated Goal Structuring Notation (GSN) assurance case synthesis dashboard; and 4) a compliance reporting dashboard. The different capabilities of RITE have been demonstrated in detail using an industrial use case – a Hybrid Electric Propulsion System (HEPS).**

***Index Terms*—Assurance Cases, Assurance Development Framework, Certification, DO-178C, ARP4754, Software and System Engineering**

I. Introduction

The assurance and certification of safety-critical airborne software and systems involve managing and evaluating a vast amount of heterogeneous evidence from diverse sources to demonstrate adherence to regulations. The evidence includes artifacts such as high-level and low-level requirements, software tests conducted, traceability of high-level requirements to low-level requirements, analysis of hazards, etc. — all of which must be holistically considered for certification. The growing complexity of modern aerospace systems will only exacerbate the costs associated with developing evidence-based assurance and compliance arguments and reports for certification, in terms of labor and time. Therefore, new tools and approaches are highly desirable to facilitate and streamline the management and evaluation of evidence to create certification arguments.

During different stages in the development life-cycle of software and systems, certification evidence is collected from various tools and sources. To show that their product complies with relevant certification standards, engineers must accurately organize heterogeneous certification evidence with respect to pre-approved plans, such as a PSAC (Plan for Software Aspects of Certification) for DO-178C [1].

In recent times, assurance cases [2] have also gained popularity for demonstrating compliance with safety goals that are necessary to get certification approval. However, due to the heterogeneous nature of the evidence and the variability in the tools and sources that produce them, coherently demonstrating compliance using plans or assurance cases in a completely automated manner is a challenging problem. This is because different tools and sources used in the system and software development process may use different terminologies and formats to represent their evidence. We present an open-source Eclipse-based Integrated Development Environment (IDE) called the RACK Integrated CerTification Environment (RITE) for software and system assurance and certification. RITE builds upon the capabilities provided by an evidence curation platform called the Rapid Assurance Curation Kit (RACK) and its semantic triple-store backed database for certification evidence curation and assurance case construction.

Several tools exist in the industry for aiding in the generation of assurance cases and compliance summaries. However, the novelty of RITE lies in its seamless integration with the back-end RACK evidence curation platform. RACK uses ontologies to normalize and curate the heterogeneous evidence for comprehensive compliance analysis. RACK has a core ontology that models the provenance of certification evidence. The RACK core ontology is written in the Semantic Application Design Language (SADL) [3], a structured English language which RITE automatically translates to Web Ontology Language (OWL). RITE is integrated with SADL so that a user can view the RACK core ontology, make extensions, generate OWL, load the ontology into RACK, all from the same environment. With such an integrated environment, users can rapidly create additional custom ontologies for various assurance case formalisms such as the Goal-Structuring Notation (GSN), for certification standards such as DO-178C [1] or

ARP4754A [4], and for domain-specific and project-specific verbiage to data curation needs.

RITE also accelerates the process of assembling evidence into a comprehensive ingestion package. It provides data curators with graphical user interfaces to manage certification evidence in several files packaged into Eclipse projects. RITE provides native support for editors for common file formats such as CSV and YAML, which are heavily used in the assembly of RACK ingestion packages.

RITE automatically synthesizes assurance cases and compliance summaries using interactive dashboards. The assurance case and compliance summaries can then be used by an engineer to check compliance status.

Finally, we demonstrate the different capabilities of RITE using a set of requirements for an industrial Hybrid-Electric Propulsion System (HEPS). Our notional architecture for HEPS comprises of multiple systems and sub-systems pertaining to mechanical, electrical and thermal domains with a top-level aircraft system decomposed into propulsion and weight-rollup subsystems. For instance, the electric drivetrain that "parallelly" delivers power to the motor consists of a sequence of power converters that perform DC-DC and DC-AC power consumption. Furthermore, there exists power controllers that monitor the state of the power converters. More details about HEPS can be found elsewhere [5]. Our notional set of HEPS requirements consists of several interconnected software and system requirements and related evidence that must be considered for safety and compliance analysis. An open-source version of notional HEPS architecture and formalized notional HEPS requirements is provided[1]. Using this notional data, we demonstrate how RITE can be used to compose project-specific extensions to RACK's core ontology, ingest the evidence, and generate assurance cases and compliance summaries using interactive and intuitive GUIs. Our example highlights the usability of RITE for the efficient curation and analysis of heterogeneous evidence for certification.

In summary, the ontology development, evidence ingestion, and reporting features make RITE a single unified environment for managing, evaluating, and presenting heterogeneous evidence needed for assurance and certification of safety-critical airborne software and systems.

Our main contributions are:

- We present RITE's capabilities to develop SADL ontologies to normalize software and system development and verification evidence.
- We introduce RITE's capabilities to ingest the normalized evidence in the RACK curation platform.
- We describe RITE's capability to automatically synthesize assurance cases in the Goal Structuring Notation (GSN) [6].
- We showcase RITE's interactive dashboards for automatically generating compliance summaries for standards like DO-178C and ARP4754A [4].

[1]https://github.com/ge-high-assurance/HEPS

The rest of the paper is organized as follows: Section II provides the background technologies required for RITE. Section III describes the related work and toolchains similar, orthogonal and complementary to RITE. Section IV describes the RITE architecture and the general workflows involved in the (automated) certification of software and system. Section V describes the detailed workflow involved in ingesting the evidence artifacts into the semantic RACK database. Section VI describes the generation of assurance cases automatically from GSN patterns formalized in SADL. Section VII presents the certification capabilities of RITE particularly in the context of DO-178C and ARP4754A. Section VIII presents an end-to-end demonstration of RITE's features using an industrial use case – a Hybrid Electric Propulsion System (HEPS) artifacts. Section IX discusses RITE's applicability in an industrial setting and its scalability with concluding summary.

II. Background

A. Semantic Application Design Language

The Semantic Application Design Language (SADL) is a declarative constrained natural language formalism to capture ontologies, knowledge graphs, and if/then rules and semantic relations that constitute a certain real-world or abstract domain [3]. SADL ontologies look very similar to English in a structure form with relational querying capabilities and explanations for the queries. SADL has been used in wide-ranging domains from additive manufacturing , material discovery, automated certification of software [7] and also in cyber security modeling [8].

B. Rapid Assurance Case Curation Kit

The Rapid Assurance Curation Kit (RACK) is a set of core ontologies expressed in SADL along with toolchains that can be used by evidence creators and evidence curators concerned with certifying complex software and systems followed by development of their assurance cases. At a fundamental level, there exists the RACK ontology which has been developed to aid the certification of evidence for civil and military airborne systems. RACK core ontologies have been extended to support evidence ingestion aimed at standards such as DO-178C, ARP475A, and MIL-STD-881D[2]. RACK can curate evidences from a diverse set of data sources from different software analysis tools such as static analyzers, model checkers and other formal methods and analyses tools and ingest their curated artifacts on to the RACK database. The RACK database is enabled by the underlying Semantics Toolkit (SemTK) technology [9] that can ingest and query semantic relations captured in OWL files and CSVs.

C. GSN Assurance Cases

Assurance cases are used to to develop complete arguments for the safety and innocuity of systems upto a certain design assurance level. Several formalisms and methodologies exist in the literature towards the development of assurance cases

[2]https://github.com/ge-high-assurance/RACK

such as the Structured Assurance Case Metamodel (SACM) [10] and the Goal Structuring Notation (GSN) [6]. GSN is a popular notation for developing assurance cases where the assurance case itself is captured as an argument tree. The top level in the tree describes the goal of safety requirement which can be broken down in to sub-goals. Each goal in the GSN tree can be demonstrated using a strategy and every strategy eventually can be traced to a solution. The solution nodes represent evidence in the GSN methodology. RITE uses the GSN formalism to generate assurance cases, but can be extended in the future to support SACM.

D. Certification of Aerospace Software

DO-178C [1] and ARP-4754A [11] are process-based industry standards used to develop and certify aerospace software to perform functions with the appropriate level of confidence for aerospace applications. The standards define a set of life-cycle processes along with objectives that need to be satisfied for each process, and activities that may be used to satisfy the objectives. The artifacts and evidence required to satisfy each objective vary based on the Development Assurance Level (DAL), which can be A through E. DO-178C calls for the creation of a Plan for Software Aspects of Certification (PSAC), that includes information about the specific evidence that will be created, to document the plan to satisfy all of the required objectives relevant to the software. PSAC is the primary means used by the certification authorities to assess whether the developed software meets the necessary criteria. When an aerospace software is a system-level software, ARP4754A may be used to certify it on the basis of a Development Assurance Plan (DAP).

E. Important Terminologies

Given below are some important terminologies that are related to RITE and will be useful to understand the data ingestion and analysis capabilities of RITE:

- **Evidence**: These are the documentation, artifacts, analysis reports, simulation data, test results, etc. generated throughout the software lifecycle that serve as evidence of assurance or compliance with certification standards.
- **Instance Data**: Normalized evidence in CSV format for ingestion.
- **Ontology**: An ontology is a formal representation of information that consists of classes and relationships between classes. They can be used to encode information in a computer-understandable manner.
- **SADL**: SADL [3] is a controlled English-like language that can be used for designing formal ontologies.
- **SemTK**: Semantics Toolkit [12] is used for storing, viewing, and querying data normalized using an ontology. SemTK constitutes the backbone of RACK's data curation capabilities.
- **Nodegroup**: An executable query that can be executed on SemTK to fetch data [13].
- **RACK's Core Ontology**: This is a formal representation of knowledge that defines the well-known entity-relationship (E-R) model representing real-world concepts or objects, and the connections between those concepts or objects. It is written in SADL and consists of classes customized to capture the evidence associated with software development and verification lifecycles.
- **Project Overlay**: RACK allows users to extend the core ontology with custom classes and relationships to capture project or domain-specific data. Such an extension to the core ontology is called an "overlay".
- **GSN Ontology**: The GSN ontology is a special overlay designed to encode GSN notation-based assurance fragments in RACK.
- **GSN Pattern**: GSN patterns are custom project-specific information that can be used by an automated tool to analyze the data inside RACK and create GSN fragments.
- **GSN Pattern Overlay**: The GSN pattern overlay is an extension of the GSN Ontology that allows users to encode project-specific GSN patterns.
- **DO-178C Ontology**: This is a special overlay that can be used for encoding a project-specific DO-178C PSAC that can be used for compliance summary generation.
- **ARP4754A Ontology**: This is a special overlay that can be used for encoding a project-specific ARP4754A DAP that can be used for compliance summary generation.
- **Ingestion**: The act of feeding data into RACK.
- **Ingestion Package**: A packaged collection of data that can be ingested into RACK. Ingestion packages have a required correct structure for successful ingestion.
- **Manifest File**: A YAML configuration file inside an ingestion package specifying ingestion steps.
- **JSON Manifest Script**: These are configuration files for the bulk ingestion of multiple ontology models, node-groups, and data (evidence) into RACK.

III. Related Work

Maksimov *et al.* provide a comprehensive list of assurance case tools in their assurance case survey paper [14]. There exists previous work on using assurance case patterns for constructing assurance case fragments. Hawkins *et al.*[15] exhibit the use of patterns to create safety cases for a prototype autonomous vehicle and an aircraft software system. Hawkins *et al.*[16] present an approach for using assurance case argument patterns along with a weaving model to link the various entities in a safety case argument structure. Taguchi *et al.*[17] present safety and security patterns that incorporate safety and security concerns at the early stages of system life-cycle. Basir *et al.*[18] present an approach for developing safety cases using safety argument templates to argue along the hierarchical structure in model-based development. Denney *et al.*[19], [20], [21], [22] present AdvoCATE, a toolset for automatic creation and management of safety cases along with hazard analysis features. AdvoCATE provides an interface for automated creation and assembly of safety case fragments, automated instantiation of safety arguments using patterns, integration of formal methods into safety arguments, and hierarchical abstraction. Resolute [23] is an assurance case

language and tool where users have to manually formulate claims and the rules for justifying them. Resolute can then generate structured assurance arguments to show if the claims can be substantiated for a specified AADL model. Wang *et al.* [24] introduce a novel SMT-based approach to automatically synthesize assurance cases based on a library of refinement relations between contracts and contract nets. The AMASS Tool Platform [25] provides a novel holistic approach to support assurance and certification for Cyber Physical Systems. When considering the dimensions along which Maksimov *et al.* [14] compare several assurance case tools, RITE's support for assurance case creation is A (strong support), support for maintaining assurance case is A, support for assessing the state of assurance case is A, support for collaboration with other developers is C (minimal support), support for creating reports from assurance cases is B (moderate support), although we have custom releases that lean towards A [26], Support for integration with requirements engineering and hazard analysis tools is currently D (no support).

IV. The RITE Functionalities and Workflow

The RITE architecture is designed to support ingestion and curation of evidential artifacts from diverse data sources using RACK, and the automated synthesis of assurance cases and reports from the artifacts. The evidential artifacts can be generated by development tools used by engineers. The artifacts can also contain provenance information such as who created the artifacts, who conducted the software test, reviews and their results, and so on. The primary functionalities offered by RITE include the following:

- Creating manifest-based ingestion packages for RACK
- Ingesting well-formed ingestion packages into RACK
- Viewing ontologies and nodegroups stored on RACK
- Quickly creating instance data loaded RACK ontologies
- Executing nodegroups and viewing results
- Generating ingestion nodegroups for the ontology currently loaded on RACK
- Developing SADL ontologies
- Creating modular ingestion packages
- Automatically synthesizing GSN assurance cases using the data inside RACK
- Automatically generating certification compliance summaries for DO-178C and ARP4754A

The RITE workflow is illustrated in Figure 1. It involves the following major steps:

1) Develop SADL ontologies, format evidential data into CSV and create JSON ingestion manifest scripts
2) Assemble ontologies, evidential data, and manifest scripts as ingestion packages with the RITE IDE
3) Ingest the evidence artifacts into the RACK database as ingestion packages created in step 2
4) Develop SADL GSN overlay, pattern ontologies, and SPARQL nodegroups for assurance cases
5) Develop SADL plans for compliance analysis.
6) Develop SPARQL nodegroups using the SemTK web interface
7) Generate and visualize assurance cases and certification reports by querying RACK

The RITE workflow begins by collecting development evidence for the system of interest, which include artifacts such as requirement documentation, design artifacts, test results, formal verification results, simulation data, etc. Upon understanding the system of interest and its development process, a project overlay is developed. Then, the developmental evidence data is formatted into CSV files, aligned with the defined overlay. The project overlay provides a structured representation of the data, facilitating data analysis later for assurance case and certification report generation. Depending on the objective, assurance case patterns or a formal certification compliance plans can be developed in SADL. They are ingested into SemTK to assist in generating assurance cases or certification reports respectively. Nodegroups are developed based on the ontologies utilizing SemTK's web interface, and are used for querying SemTK for fetching data to construct the assurance cases and certification reports. Then, JSON manifest files are developed for the ease of bulk ingestion of multiple models, nodegroups, and data, which provide step-by-step instructions on how the data should be loaded into SemTK. To consolidate the information, the ontologies including core, overlay, assurance case patterns, SADL plans, and evidential data are compiled into an ingestion package, which can be subsequently ingested to SemTK as dictated by a set of manifest files. Overall, this workflow ensures a systematic collection, organization, and ingestion of development evidence. The evidential data from several sources are appropriately curated by SemTK using the RACK core and overlay ontologies. Post-ingestion, RITE provides the means to generate GSN assurance cases and certification compliance reports by querying appropriate information from the RACK database using pre-developed and synthesized nodegroups.

V. Artifact Data Ingestion and Retrieval

A. RACK Ingestion Packages

RACK's resource description framework (RDF) properties allows users to store unstructured artifact data in a common repository in the form of triples. To support data triples, RACK requires that a user specifies an artifact model (or class), a method for instance data to be ingested (mapped to a model) and the instance data itself. Each of these RACK components must be in sync with the others for data to be added. These components also have dependencies on each other and must be deployed to RACK in a specific order. The ingestion package standards were created to enable users to bundle all RACK component files within a single directory. In combination with RITE, ingestion packages offer a convenient way to distribute *"snapshots"* of RACK components which can be modified by other data or users. This becomes especially valuable for one of the main features of RACK: the ability to easily expand ontologies and datasets with new artifacts, artifact relationships, and queries from disjointed domain knowledge, schemas, and data sources.

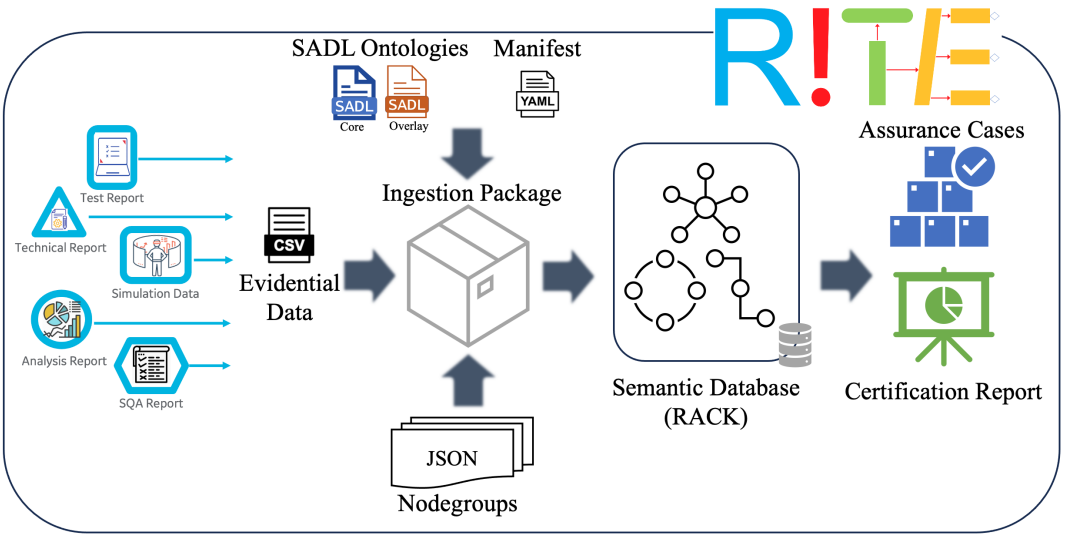

Fig. 1. RITE IDE workflow.

B. Ingestion Package Project Structure

At the root of every RACK ingestion package is a project manifest.yaml file as shown in Figure 3. This file contains descriptors for the project along with a list of all components and their relative file locations to be included in the ingestion package. A typical manifest would include a footprint (URIs for data and model graph databases) and deployment steps. Deployment steps might contain a list of children manifests, nodegroups for data insertion and querying, models representing artifact structures, and artifact instance data and copy-graph deployment steps. Components should typically be listed in the specified order to satisfy the dependencies of each component (*e.g.,* ingestion nodegroups and models are required prior to adding data).

There are two yaml files to configure the ingestion project. The first yaml file is data and ontology mapping file named data.yaml as shown in Figure 2 contain a list of ingestion steps which map instance data .csv files within the same directory to a designated model class. During processing, each row of instance data will correlate to an entity of the designated class. The second one is ingestion configuration manifest file as shown in Figure 3, wherein the *name* and *description* fields are informational, and the *footprint* are the model and data graphs to the ontology and instance data. The *steps* section is required as it describes the sequential process of loading this ingestion package.

Common types of steps that can be specified in a manifest:

- *manifest* steps take a relative path argument and recursively import that manifest file.
- *model* steps take a relative path argument and import ontology models on that file.
- *nodegroups* steps take a relative path argument and import nodegroups in that directory.
- *data* steps take a relative path argument and import data in that directory.

```
data-graph: http://rack001/data
ingestion-steps:
- class: http://arcos.rack/AircraftItem#Item
  csv: AircraftItem_Item.csv
- class: http://arcos.rack/AircraftItem#ItemReq
  csv: AircraftItem_ItemReq.csv
- class: http://arcos.rack/AircraftItem#ItemReq
  csv: AircraftItem_ItemReq_addl.csv
- class:
   http://arcos.rack/AircraftSystem#Aircraft
  csv: AircraftSystem_Aircraft.csv
- class:
   http://arcos.rack/AircraftSystem#AircraftReq
  csv: AircraftSystem_AircraftReq.csv
```

Fig. 2. Data and ontology mapping data.yaml

```
name: Hybrid Electric Propulsion System (HEPS)
description: Instance data of our HEPS
footprint:
  model-graphs:
  - http://rack001/model
  data-graphs:
  - http://rack001/data
steps:
- model: OwlModels/model.yaml
- model: GSN-Ontology/OwlModels/import.yaml
- model: HEPS-GSN-Pattern/OwlModels/import.yaml
- data: data/import.yaml
- nodegroups: nodegroups
```

Fig. 3. Ingestion configuration manifest file

C. Ingestion Package Export Workflows

After an ingestion package has been created (or imported) a user is able to upload the package and inspect all components (nodegroup, ontology, and instance data) within a hosted RACK instance. Configuration available in RITE allows a user to connect to a hosted instance of RACK. When a RACK instance is connected, the user can choose to upload entire

ingestion packages or individual RACK components using RITE menu options. The context menu of a RACK ingestion package (from the Eclipse package explorer) exposes the options to "Upload as Ingestion Package" as well as "Upload Nodegroups". Selecting "Upload as Ingestion Package" with a selected directory will prompt the user for a save location where the generated ingestion package will be saved. The resulting package (.zip) can be distributed to other developers or uploaded to a version-controlled repository and re-uploaded using the same context menu ingestion workflow. After the ingestion package is saved locally, it is uploaded to the RACK instance and RACK logging and errors are transmitted to the RITE console for debugging.

After an ingestion package has been uploaded, RITE offers various views to inspect RACK components. The RACK ontology view provides insight on classes and class instance metrics. The nodegroup store view provides a list of available ingestion and query nodegroups which can be executed to open the query results pane, a table revealing live instance data.

VI. Assurance Case Synthesis

RITE supports the automatic generation of assurance case fragments in the GSN notation from the data stored inside RACK. To allow instantiation of such GSN elements in RACK, we have formalized a GSN core ontology in SADL by integrating it with the RACK core ontology. The GSN core ontology allows formalizing the verbiage of the GSN Community Standard V2 [27] and also supports additional metadata in the form of colored GSN fragments. However, to make meaningful GSN fragments from RACK data, it is necessary to be able to connect the data with GSN terminologies such as goals, strategies, and contexts. To do this automatically, an algorithm requires access to domain-knowledge necessary to make such inferences. The GSN pattern overlay is an extension of the GSN ontology that can be used to encode such domain knowledge as custom project-specific *GSN patterns*. We have designed an GSN Inference Engine (GIEn) as a part of RITE that can automatically query RACK for available evidence and use these project-specific GSN patterns to synthesize GSN fragments using the evidence. The GSN pattern overlay has several types of patterns for representing *connections* (information about domain-specific GSN elements) and *annotations* (metadata used by the GIEn to interpret the patterns). The visualized GSN fragments are shown in Figure 4.

Fig. 5 shows the schematics of RITE's GSN synthesis workflow. It involves the following steps:

1) First, the domain experts create GSN patterns for their project using the GSN pattern overlay.
2) Then, the evidence and the project-specific patterns are ingested into RACK.
3) The users then have to open the Assurance Case interface trigger "Automatic GSN Inference". This triggers the GIEn which fetches the data from RACK and populates the screen with a list of goals for which GSN fragments can be generated.
4) Users then select a goal and triggers GIEn to automatically synthesizes a GSN fragment for the selected goal.

Apart from automatic GSN inference, RITE also provides an interactive interface that can allow users to conveniently navigate a GSN tree by drilling up or down the tree hierarchy. The interface provides useful metrics at each level that make it easy to understand the evidence-based status of a goal. Section VIII provides an example of RITE's GSN capabilities.

VII. Certification Compliance Summary Synthesis

The development and verification lifecycle of critical aerospace software generates a vast amount of data from different tools and sources. For certification, it is necessary to show compliance to appropriate standards such as DO-178C or ARP4754A, depending on the context. The complexity and heterogeneity of the data make it challenging to keep track of compliance status in a tool-agnostic manner while engineers generate and evaluate the data during the software development and verification lifecycle. To address these challenges, RITE supports a holistic approach that allows periodic tracking and review of compliance throughout the Federal Aviation Administration (FAA) stages of involvement (SOI).

To check compliance to a standard such as DO-178C or ARP4754A, engineers first have to create a plan (PSAC or DAP, see Section II-D) and then show that the evidence supports the relevant objectives in the plan. The DO-178C or ARP4754A ontologies can be used to create a formal version of a plan in SADL. Users can create and ingest such plans from directly within RITE's SADL editor and ingestion package creation interfaces. Once a plan has been ingested into RACK along with necessary evidence, RITE can be used to automatically synthesize compliance summaries using an interactive dashboard that can show the status of the evidence created for each of the objectives in the plan. Using the dashboard, users can efficiently and conveniently review the evidence to understand the status of compliance in terms of the satisfaction of relevant objectives based on DAL and other criteria. The tool also allows the user to drill down into the evidence to see, for example, where tests have passed or failed, or reviews have been created. Development programs typically run for years and generate hundreds of thousands of artifacts and evidence items. RITE's compliance dashboard enables users to efficiently curate and review the evidence to support the complex development and certification of safety critical aerospace software.

RITE supports the synthesis of compliance summaries for DO-178C and ARP4754A. We use DO-178C as an example to illustrate the concept here. The DO-178C compliance summary synthesis feature (Fig. 6) involves the following steps:

1) First, engineers decide on a DO-178C PSAC for their software and then develop a formal instance of their PSAC in SADL using the DO-178C Ontology.
2) The evidence and the formal PSAC are then added to an ingestion package and ingested into RACK.
3) The users then have to open the DO-178C Compliance Summary interface of RITE and click on a button. This

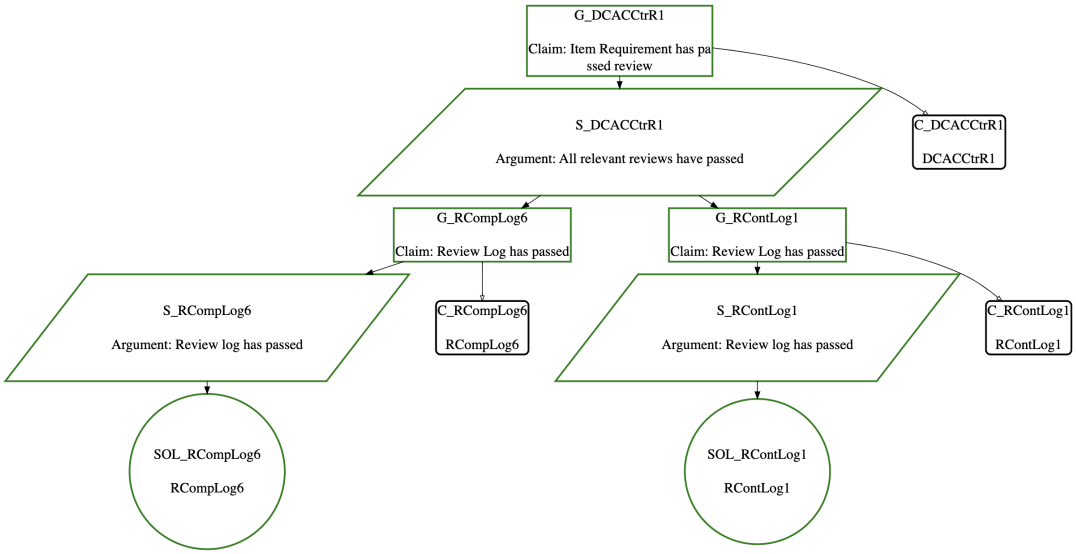

Fig. 4. A sample assurance case synthesized by RITE.

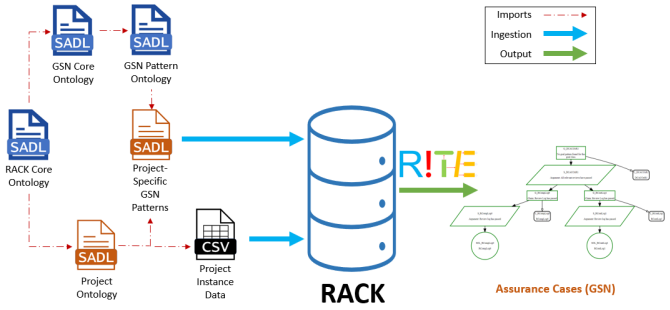

Fig. 5. Assurance case generation workflow.

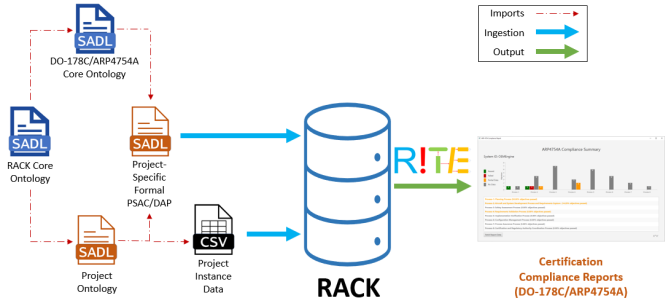

Fig. 6. Compliance summary generation workflow.

triggers the tool to automatically fetch the PSAC and the evidence from RACK, compare the evidence with the objectives in the PSAC, and populate the interactive dashboard (Fig. 7).

4) Users can then use the interactive dashboard to check the compliance at three different levels - PSAC level, Table level, and Objective level. At each level, customized metrics are generated to make it easy to evaluate the compliance status. Users can drill up and down between the different levels using the interactive GUI. More detailed information about RITE's PSAC compliance summaries can be found elsewhere [28].

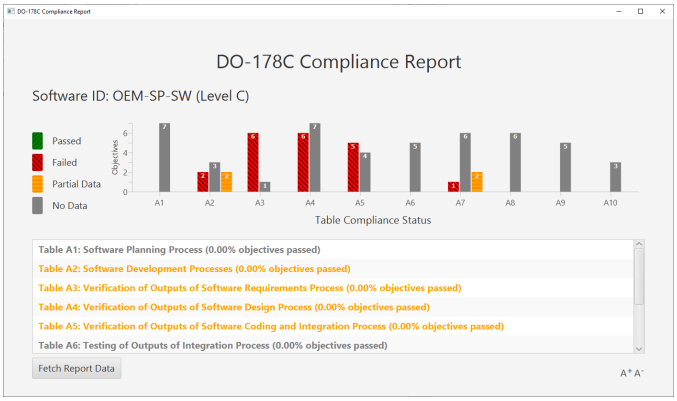

Fig. 7. The interactive DO-178C dashboard.

A detailed example of the ARP5754A compliance summary synthesis feature of RITE has been provided in Section VIII.

VIII. Use Case Study

This section provides an end-to-end demonstration of RITE's capabilities using an industrial HEPS (Hybrid Electric Propulsion System) project. HEPS is an architecture in which the traditional gas turbine engine power of an aircraft is supplemented by a battery-powered electric motor to provide the required power. We use a simulated dataset for a HEPS controls software system in this example.

A. Preparing and Uploading Data into RACK

The ingestion step involves collecting all the data that must be made available inside RACK in the form of a self-contained *ingestion package*. The ingestion package contains things such

as ontologies, formal plans, and GSN patterns written in SADL, and evidential data compiled as CSV. We present some examples of these elements for the HEPS project and describe the ingestion process.

To curate evidence inside RACK, it is necessary to develop custom ontologies to capture project-specific evidence. Fig. 8 shows snippet from an ontology that we created for the HEPS project to capture evidence associated with software and system development lifecycles in accordance with ARP4754A.

```
uri "http://arcos.rack/AircraftItem" alias AcItem.

import "http://arcos.rack/AircraftSystem".

Item is a type of SYSTEM.
itemComponent describes {SYSTEM or Item} with values of type Item.

partOf_Item describes Item with values of type Item.

ItemReq is a type of REQUIREMENT.
ItemDerivedReq is a type of REQUIREMENT.
governs of {ItemReq or ItemDerivedReq} only has values of type Item.
satisfies of ItemReq only has values of type {SystemReq or ItemReq}.

RequirementCompleteCorrectReview is a type of REVIEW.
Rv:governedBy of RequirementCompleteCorrectReview only has values of type
        {SystemReq or SystemDerivedReq or ItemReq or ItemDerivedReq}.

RequirementTraceableReview is a type of REVIEW.
Rv:governedBy of RequirementTraceableReview only has values of type
{SYSTEM or Item}.
```

Fig. 8. Snippet from the HEPS ontology.

To synthesize GSN fragments from RACK data, users must specify project-specific GSN patterns. Fig. 9 shows snippet from an example pattern that specifies goals, strategies, and evidence classes from the HEPS ontology to use for creating the GSN fragments.

```
//-- Goal patterns
 itemReq-GP is a Goal
        with isPat true
        with identifier "itemReq-GP"
        with description "Claim: Item Requirement has passed review"
        with pGoal "ENTITY".

//-- Strategy patterns
 itemReq-SP is a Strategy
        with isPat true
        with identifier "itemReq-SP"
        with description "Argument: All relevant reviews have passed"
        with pGoal "ENTITY"
        with pSubGoal "REVIEW_LOG"
        with pGoalSubGoalConnector "reviews".

//-- Evidence patterns
 result-evdnc is a GsnEvidence
        with classId "REVIEW_STATE"
        with statusProperty "identifier"
        with passValue "Passed".
```

Fig. 9. GSN patterns designed for the HEPS project.

The reporting features of RITE can fetch the evidence inside RACK, compare it against a formal plan (PSAC or DAP) and populate reports to show compliance using interactive dashboards. Fig. 10 shows a snippet from an example HEPS DAP that specifies the objectives and processes to consider for compliance checking. The ingestion package contains the project evidence in the form of CSV files. Fig. 11 shows sample evidence for the HEPS project where requirements have been defined and traced to one another. Examples of elements in HEPS ingestion package have been provided in Fig. 2 and Fig. 3.

```
HEPS is a SYSTEM
        with identifier "HEPS"
        with description "HEPS Use Case"
        with developmentAssuranceLevel LevelA.

 //-- The DAP
 HEPS-DAP is a DevelopmentAssurancePlan
        with identifier "HEPS-DAP"
        with description "Development Assurance Plan for the HEPS Project"
        with system HEPS
        with process Process-1
        with process Process-2
        with process Process-3
        with process Process-4
        with process Process-5
        with process Process-6
        with process Process-7
        with process Process-8.

 //-- Objective 2-2 info
 Objective-2-2 has query "Objective-2-2-query-count-all-System-Requirements".
 Objective-2-2 has query "Objective-2-2-query-count-all-System-Requirements-allocated-
to-a-system".
 Objective-2-2 has query "Objective-2-2-query-count-all-Interfaces-allocated-to-a-
system".
```

Fig. 10. A snippet from the DAP for the HEPS project.

```
identifier,dataInsertedBy_identifier,description,title
DCDCCtrR5,Cameo Ingestion,DCDCcontroller shall set
↪  Fault_DCDC of DCDCcontroller to 1.0 when Temp_bat
↪  of battery > max_temp_bat,DCDCController,,
DCDCCtrR6,Cameo Ingestion,DCDCcontroller shall set
↪  Fault_DCDC of DCDCcontroller to 1.0 when
↪  Temp_DCDC of DCDCcontroller >
↪  Max_temp_mod,DCDCController,,
DCDCCtrR7,Cameo Ingestion,DCDCcontroller shall set
↪  Fault_DCDC of DCDCcontroller to 1.0 when
↪  DESAT_DCDC of DCDCcontroller is
↪  1.0,DCDCController,,
```

Fig. 11. Sample CSV data for HEPS requirements.

B. Generating Reports and Assurance Cases

Once the data has been ingested inside RACK, users can generate assurance cases and/or compliance summaries.

Upon selecting an "Automatic GSN Inference" function, a dashboard opens up that allows users to populate all possible GSN goals from RACK (Fig. 12). It lists the different goal classes and also provides a graph showing the distribution of the goals or claims by different evidence classes in the HEPS ontology. Users can select a goal and trigger "Generate artifacts" to generate GSN artifacts as shown in Fig. 4. Once the GSN artifacts are generated, the dashboard provides an option to open a navigator view (Fig. 12) which allows traversal of the GSN tree in a recursive manner while showing useful metrics about the claim in focus at every level. Users can drill down a GSN tree into the subgoals navigate to previuosly explored goals using "Previous Goal" function. The navigator view also provides an interactive cascading view of the GSN where the goal in focus is highlighted in yellow, making it easier to know one's position w.r.t the root goal.

Similar to "Automatic GSN Inference", interactive dashboards that can give a summary of compliance at a quick glance can be launched (*E.g.*, Fig 7). Users can then drill-down until details of the evidence inside RACK can be seen along with useful metrics that provide a status of evidences

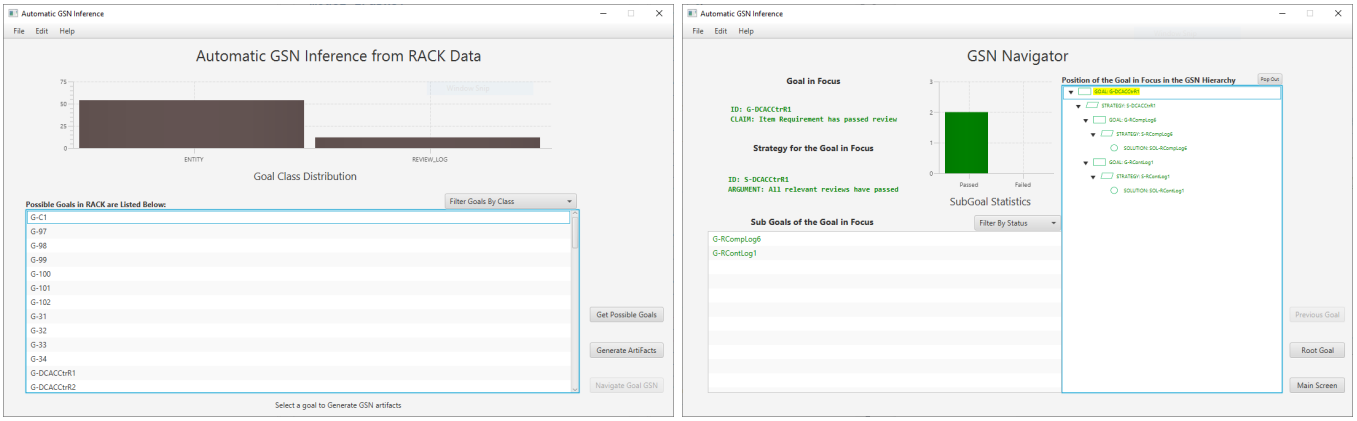

Fig. 12. RITE's GSN generator showing all possible goals (left) and navigator view (right).

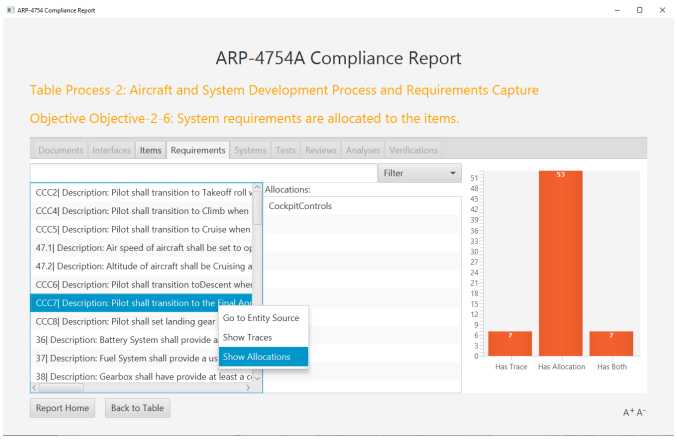

Fig. 13. Objective view of RITE's ARP4754A Compliance Summary.

relevant to a plan objective (*E.g.*, Fig. 13). The dashboard currently only supports DO-178C and ARP4754A compliance summaries. More details about traceability features in ARP4754A compliance summary can be found elsewhere [26].

IX. Discussion and Conclusion

We have presented the many features of RITE and have given an end-to-end demonstration of the tool on an industrial propulsion use case. While this work was the result of a government research program, as an industrial research center our focus is to bring innovative yet practical solutions and show how they can be applied to real-world problems on an industrial scale. This section is a discussion of the productivity improvements for the many various people involved in preparing data for certification by having a tool like RITE that incorporates automated software engineering features.

In partnership with aerospace colleagues that have gone through the certification process on multiple programs, we identified the following distinct personas with unique responsibilities: a technical project manager, a data curator, a certification compliance engineer, and an assurance engineer. Each of them were assigned dedicated tasks but all of them needed access to the breadth of data using a common platform. This is where RITE brings productivity improvements.

A technical project manager is responsible for formalizing a PSAC or DAP. They can do this using RITE by developing ontologies in SADL. RITE provides syntactic and semantic validation via semantic coloring, model data type checking, hyperlinking, and content auto-complete. They can save the formalized plan(s) in a shareable ingestion package.

The data curator's job is to organize, sort, and manage the development data that gets generated throughout the life cycle of a system under development. The plans written by the technical project manager serves as a guideline and template for how software and system development data are organized, ingested, and retrieved for certification and assurance purposes. RITE helps with the data curator's productivity by generating ontology-compliant CSV templates with headers. Furthermore, RITE handles modular ingestion packages so the data curator can add data from new artifacts while the project is still undergoing development.

The certification compliance engineer's job is to compare the project artifacts against objectives in the PSAC and stay up to date with compliance statistics. RITE's dashboard, with the press of a single button, fetches the PSAC and the evidence from RACK, compares the evidence against the objectives, and generates an interactive report with meaningful statistics.

An assurance engineer can use RITE to automatically generate assurance case fragments in the form of GSN from the data stored in RACK. Because the GSN patterns are formalized, the workflow in RITE saves the assurance engineer time by easily updating their assurance cases even as certification data is being generated daily.

In addition to providing productivity gains, RITE is also scaleable as the underlying SemTK technology supports configuration of different triplestores such as: Fuseki [29], an open-source RDF framework; AllegroGraph [30], designed to handle large volumes of data, available via a commercial licensing structure. Project teams select and size a triplestore appropriately based on project needs and budgets.

In summary, we introduced an Eclipse-based tool – RITE

for the assurance and certification of software and systems. It offers researchers and practitioners a powerful framework and streamlined process to organize system development artifacts, ingest artifacts into a semantic database, and retrieve data for meaningful representation such as assurance case and certification. We have delved into the various features and capabilities of RITE and provide illustrative examples for each functionality. These functionalities, combined with an intuitive GUI and comprehensive documentation, make the tool accessible to the community. The RITE tool, example models and demos are publicly available on GitHub. [3]. As part of future work, RITE can support Large-Language Model assisted ontology-development and ingestion package creation workflows to accelerate data curation. This can potentially offset our aforementioned personas due to potential gains in efficiency of data curation with the help of LLMs. RACK has been used in conjunction with version management, tool configuration management frameworks such as Evidential Toolbus (ETB2) for continuous evidence curation and assurance case construction [31]. As part of future work, RITE can be extended to support RACK-based ETB2 workflows.

Acknowledgement & Disclaimer. Distribution Statement "A" (Approved for Public Release, Distribution Unlimited). This research was developed with funding from the Defense Advanced Research Projects Agency (DARPA). The views, opinions and/or findings expressed are those of the author and should not be interpreted as representing the official views or policies of the Department of Defense or the U.S. Government.

REFERENCES

[1] RTCA, "DO-178C Software Considerations in Airborne Systems and Equipment Certification."

[2] Verocel Inc., "VeroTrace," 2023. [Online]. Available: https://www.verocel.com/aerospace/do-178-certification/

[3] A. Crapo and A. Moitra, "Toward a unified English-like representation of semantic models, data, and graph patterns for subject matter experts," *International Journal of Semantic Computing*, vol. 7, no. 03, pp. 215–236, 2013.

[4] SAE, "ARP-4754A Guidelines for Development of Civil Aircraft and Systems," 2010.

[5] P. Wheeler, T. S. Sirimanna, S. Bozhko, and K. S. Haran, "Electric/hybrid-electric aircraft propulsion systems," *Proceedings of the IEEE*, vol. 109, no. 6, pp. 1115–1127, 2021.

[6] T. Kelly and R. Weaver, "The goal structuring notation–a safety argument notation," in *Proceedings of the dependable systems and networks 2004 workshop on assurance cases*. Citeseer, 2004, p. 6.

[7] A. Moitra, P. Cuddihy, K. Siu, D. Archer, E. Mertens, D. Russell, K. Quick, V. Robert, and B. Meng, "RACK: A Semantic Model and Triplestore for Curation of Assurance Case Evidence," in *10th International Workshop on Next Generation of System Assurance Approaches for Safety-Critical Systems (SASSUR), Toulouse, France, September, Computer Safety, Reliability, and Security (SAFECOMP), Lecture Notes in Computer Science*, 2023.

[8] B. Meng, D. Larraz, K. Siu, A. Moitra, J. Interrante, W. Smith, S. Paul, D. Prince, H. Herencia-Zapana, M. F. Arif *et al.*, "Verdict: a language and framework for engineering cyber resilient and safe system," *Systems*, vol. 9, no. 1, p. 18, 2021.

[9] P. Cuddihy, J. McHugh, J. W. Williams, V. Mulwad, and K. S. Aggour, "SemTK: An ontology-first, open source semantic toolkit for managing and querying knowledge graphs," *arXiv preprint arXiv:1710.11531*, 2017.

[10] R. Wei, T. P. Kelly, X. Dai, S. Zhao, and R. Hawkins, "Model based system assurance using the structured assurance case metamodel," *Journal of Systems and Software*, vol. 154, pp. 211–233, 2019.

[11] S-18 Aircraft and Sys Dev and Safety Assessment Committee, *Guidelines for development of civil aircraft and systems*. SAE International, 2023.

[12] P. Cuddihy, J. McHugh, J. W. Williams, V. Mulwad, and K. S. Aggour, "Semtk: A semantics toolkit for user-friendly sparql generation and semantic data management." in *ISWC (P&D/Industry/BlueSky)*, 2018.

[13] V. Kumar, P. Cuddihy, and K. S. Aggour, "NodeGroup: a knowledge-driven data management abstraction for industrial machine learning." in *Proceedings of the 3rd International Workshop on Data Management for End-to-End Machine Learning*, 2019, pp. 1–4.

[14] M. Maksimov, N. L. Fung, S. Kokaly, and M. Chechik, "Two decades of assurance case tools: a survey," in *Computer Safety, Reliability, and Security: SAFECOMP 2018 Workshops, ASSURE, DECSoS, SASSUR, STRIVE, and WAISE, Västerås, Sweden, September 18, 2018, Proceedings 37*. Springer, 2018, pp. 49–59.

[15] R. Hawkins, K. Clegg, R. Alexander, and T. Kelly, "Using a software safety argument pattern catalogue: Two case studies," in *International Conference on Computer Safety, Reliability, and Security*. Springer, 2011, pp. 185–198.

[16] R. Hawkins, I. Habli, D. Kolovos, R. Paige, and T. Kelly, "Weaving an assurance case from design: a model-based approach," in *2015 IEEE 16th International Symposium on High Assurance Systems Engineering*. IEEE, 2015, pp. 110–117.

[17] K. Taguchi, D. Souma, and H. Nishihara, "Safe & sec case patterns," in *International Conference on Computer Safety, Reliability, and Security*. Springer, 2014, pp. 27–37.

[18] N. Basir, E. Denney, and B. Fischer, "Deriving safety cases for hierarchical structure in model-based development," in *International Conference on Computer Safety, Reliability, and Security*. Springer, 2010, pp. 68–81.

[19] E. Denney and G. Pai, "Automating the assembly of aviation safety cases," *IEEE Transactions on Reliability*, vol. 63, no. 4, pp. 830–849, 2014.

[20] ——, "A methodology for the development of assurance arguments for unmanned aircraft systems," in *33rd International System Safety Conference (ISSC 2015)*, 2015.

[21] E. Denney, G. Pai, and J. Pohl, "AdvoCATE: An assurance case automation toolset," in *International Conference on Computer Safety, Reliability, and Security*. Springer, 2012, pp. 8–21.

[22] E. Denney and G. Pai, "Tool support for assurance case development," *Automated Software Engineering*, vol. 25, no. 3, pp. 435–499, 2018.

[23] A. Gacek, J. Backes, D. Cofer, K. Slind, and M. Whalen, "Resolute: an assurance case language for architecture models," *ACM SIGAda Ada Letters*, vol. 34, no. 3, pp. 19–28, 2014.

[24] T. E. Wang, Z. Daw, P. Nuzzo, and A. Pinto, "Hierarchical contract-based synthesis for assurance cases," in *NASA Formal Methods Symposium*. Springer, 2022, pp. 175–192.

[25] J. L. De La Vara, E. Parra, A. Ruiz, and B. Gallina, "The amass tool platform: An innovative solution for assurance and certication of cyber-physical systems." in *REFSQ Workshops*, 2020.

[26] S. Paul, K. Siu, B. Meng, M. Durling, D. Prince, and C. McMillan, "A Semantic Triplestore-Based ARP4754ACompliance Summary Dashboard," in *43rd IEEE/AIAA 42nd Digital Avionics Systems Conference (DASC)*, 2024.

[27] A. C. W. Group *et al.*, "GSN community standard version 2," *Available on-line: https://scsc. uk/scsc-141B*, 2018.

[28] S. Paul, C. Alexander, M. Durling, K. Siu, D. Prince, B. Meng, S. C. Varanasi, and D. Stuart, "Automated DO-178C Compliance Summary through Evidence Curation," in *42nd IEEE/AIAA 42nd Digital Avionics Systems Conference (DASC)*, 2023, pp. 1–10.

[29] Apache Jena, "Fuseki," 2024. [Online]. Available: https://jena.apache.org/documentation/fuseki2/

[30] Franz, Inc., "AllegroGraph," 2024. [Online]. Available: https://allegrograph.com/

[31] N. Shankar, M. Kim, H. Sanchez, H. Rueß *et al.*, "Continuous safety & security evidence generation, curation and assurance case construction using the evidential tool bus," in *2024 AIAA DATC/IEEE 43rd Digital Avionics Systems Conference (DASC)*, 2024, pp. 1–10.

[3] https://github.com/ge-high-assurance/RITE